


# INTRODUCTION TO PROGRAMMING USING JAVA LAB (BCS351)

## B.Tech CSE 2nd SEMESTER

**SUBMITTEDTO:**

**MR.** UNWAN KHAN

**SUBMITTED BY:**

Aditya Singh
BCS2025353

# INDEX

**Name: Nitin Kumar** **Roll No: BCS2025601** **Section: "B"**


| S.N. | PROGRAM NAME | PAGE | DATE | SIGNATURE |
|---|---|---|---|---|
| 1 | Hello World, Variables and Arithmetic | 3 | | |
| 2 | Decisions and Loops with Numbers | 6 | | |
| 3 | Star and Number Patterns | 9 | | |
| 4 | Writing Functions | 12 | | |
| 5 | Arrays: Traversal and Counting | 15 | | |
| 6 | Arrays: Reverse and Duplicates | 18 | | |
| 7 | Counting Steps and Complexity | 21 | | |
| 8 | Matrices: Transpose and Row Sums | 24 | | |
| 9 | Matrices: Rows and Columns | 27 | | |
| 10 | Switch and ArrayList | 30 | | |
| 11 | Bubble Sort and Selection Sort | 33 | | |
| 12 | Insertion Sort | 36 | | |
| 13 | Strings: Methods and Palindromes | 39 | | |
| 14 | Characters: Counting and Positions | 42 | | |
| 15 | StringBuilder Methods | 45 | | |

# EXPERIMENT NO. 1

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Hello World, Variables and Arithmetic

**2. Aim:**

- To print text, store values in variables and calculate with arithmetic operators.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- A Java program starts execution from the main() method. System.out.println() prints text and moves to the next line.
- A variable stores a value in memory. Java has primitive types such as int (whole numbers), double (decimal numbers) and char (single character), and the reference type String for text.
- Arithmetic operators are + (add), - (subtract), * (multiply), / (divide) and % (remainder). Dividing two int values gives an int result. A constant is declared with the final keyword and cannot be changed after it is assigned.

**5. Algorithm / Procedure:**

- **Task 1: Print Hello World on one line and Welcome to Java on the next line.**

1. Start the program and enter main().
2. Print "Hello World" using println().
3. Print "Welcome to Java" using println().
4. Stop.

- **Task 2: Create roll, percentage, section and name variables and print each with its label.**

1. Declare int roll = 101, double percentage = 84.5, char section = 'B', String name = "Asha".
2. Print each label followed by its variable on its own line.
3. Stop.

- **Task 3: Perform sum, difference, product, quotient and remainder on a = 17 and b = 5.**

1. Declare int a = 17 and int b = 5.
2. Calculate a + b, a - b, a * b, a / b and a % b.
3. Print each result after its label.
4. Stop.

- **Task 4: Use a constant PI (final) to find the area and perimeter of a circle of radius 7.**

1. Declare final double PI = 3.14 and int radius = 7.
2. Area = PI * radius * radius.
3. Perimeter = 2 * PI * radius.
4. Print Area and Perimeter.
5. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println("Welcome to Java");
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        int roll = 101;
        double percentage = 84.5;
        char section = 'B';
        String name = "Asha";

        System.out.println("Roll: " + roll);
        System.out.println("Percentage: " + percentage);
        System.out.println("Section: " + section);
        System.out.println("Name: " + name);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int a = 17;
        int b = 5;

        System.out.println("Sum: " + (a + b));
        System.out.println("Difference: " + (a - b));
        System.out.println("Product: " + (a * b));
        System.out.println("Quotient: " + (a / b));
        System.out.println("Remainder: " + (a % b));
    }
}
```

**Task 4**

```
public class Main {
    public static void main(String[] args) {
        final double PI = 3.14;
        int radius = 7;
        double area = PI * radius * radius;
        double perimeter = 2 * PI * radius;
        System.out.printf("Area: %.2f%n", area);
        System.out.printf("Perimeter: %.2f%n", perimeter);
    }
}
```

## 7. Input:

- **Task 1:** No keyboard input. Test data is fixed inside the program (given below).
- **Task 2:** roll = 101, percentage = 84.5, section = 'B', name = "Asha"
- **Task 3:** a = 17, b = 5
- **Task 4:** PI = 3.14 (constant), radius = 7

## 8. Output:

**Task 1**

```
Hello World
Welcome to Java
```

**Task 2**

```
Roll: 101
Percentage: 84.5
Section: B
Name: Asha
```

**Task 3**

```
Sum: 22
Difference: 12
Product: 85
Quotient: 3
Remainder: 2
```

**Task 4**

```
Area: 153.86
Perimeter: 43.96
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is the use of the main() method in Java?
Q2. What is the difference between print() and println()?
Q3. Why is 17 / 5 equal to 3 and not 3.4 when both are int?
Q4. What does the final keyword do when used with a variable?
Q5. Name the primitive data types used in this experiment.

# EXPERIMENT NO. 2

Date of Performancer:- Course: BCS351

## 1. Experiment Title:

- Decisions and Loops with Numbers

## 2. Aim:

- To use if, else if and loops to make decisions and repeat steps with whole numbers.

## 3. Software / Tools Required:

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

## 4. Theory:

- The if-else statement chooses between two paths depending on a condition. An else-if ladder checks several conditions one after another and runs the first one that is true.
- The remainder operator % is used to test divisibility: n % 2 == 0 means n is even.
- A for loop repeats a block of code a fixed number of times using an initialisation, a condition and an update. Factorial of n is the product 1 x 2 x ... x n.
- A leap year is divisible by 4 but not by 100, or it is divisible by 400.

## 5. Algorithm / Procedure:

**Task 1: Check whether n = 27 is even or odd.**

1. Store 27 in n.
2. If n % 2 == 0 print "is even", else print "is odd".
3. Stop.

**Task 2: Print the grade for marks = 83 using an else-if ladder.**

1. Store 83 in marks.
2. If marks >= 90 grade is A; else if marks >= 75 grade is B; else grade is C.
3. Print the grade.
4. Stop.

**Task 3: Check whether year = 2024 is a leap year.**

1. Store 2024 in year.
2. If (year % 4 == 0 and year % 100 != 0) or year % 400 == 0, it is a leap year.
3. Print the result.
4. Stop.

**Task 4: Find the factorial of 5 using a for loop.**

1. Set fact = 1.
2. Repeat for i = 1 to 5: fact = fact * i.
3. Print the value of fact.
4. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        int n = 27;
        if (n % 2 == 0) {
            System.out.println(n + " is even");
        } else {
            System.out.println(n + " is odd");
        }
    }
}
```

### Task 2

```
public class Main {
    public static void main(String[] args) {
        int marks = 83;
        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else {
            System.out.println("Grade: C");
        }
    }
}
```

### Task 3

```
public class Main {
    public static void main(String[] args) {
        int year = 2024;
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            System.out.println("Leap year");
        } else {
            System.out.println("Not a leap year");
        }
    }
}
```

### Task 4

```
public class Main {
    public static void main(String[] args) {
        int fact = 1;
        for (int i = 1; i <= 5; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial of 5 is " + fact);
    }
}
```

## 7. Input:

- **Task 1:** n = 27
- **Task 2:** marks = 83
- **Task 3:** year = 2024
- **Task 4:** Loop runs for i = 1 to 5 (factorial of 5)

## 8. Output:

**Task 1**

```
27 is odd
```

**Task 2**

```
Grade: B
```

**Task 3**

```
Leap year
```

**Task 4**

```
Factorial of 5 is 120
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is the difference between if-else and an else-if ladder?
Q2. How do you check whether a number is even using the % operator?
Q3. Why is 1900 not a leap year but 2000 is?
Q4. Write the three parts of a for loop.
Q5. What is the factorial of 0?

# EXPERIMENT NO. 3

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Star and Number Patterns

**2. Aim:**

- To use a loop inside another loop (nested loops) to print stars and numbers in rows.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- A nested loop is a loop placed inside another loop. For every single run of the outer loop, the inner loop runs completely.
- In pattern printing the outer loop usually controls the rows and the inner loop controls the columns (the number of stars or numbers in each row).
- System.out.print() prints without moving to a new line, and System.out.println() moves to the next line. A println() after the inner loop ends the row.

**5. Algorithm / Procedure:**

**Task 1: Print a 4 x 4 square of stars.**

1. Outer loop i runs from 1 to 4 (rows).
2. Inner loop j runs from 1 to 4 and prints "*".
3. After the inner loop print a new line.
4. Stop.

**Task 2: Print a right triangle of stars with 4 rows.**

1. Outer loop i runs from 1 to 4.
2. Inner loop j runs from 1 to i and prints "*".
3. Print a new line after each row.
4. Stop.

**Task 3: Print a number triangle with 4 rows.**

1. Outer loop i runs from 1 to 4.
2. Inner loop j runs from 1 to i and prints j followed by a space.
3. Print a new line after each row.
4. Stop.

**Task 4: Print an upside-down triangle of stars with 4 rows.**

1. Outer loop i runs from 4 down to 1.
2. Inner loop j runs from 1 to i and prints "*".
3. Print a new line after each row.

4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 4; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

**Task 4**

```
public class Main {
    public static void main(String[] args) {
        for (int i = 4; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

## 7. Input:

- **Task 1:** 4 rows and 4 columns
- **Task 2:** 4 rows
- **Task 3:** 4 rows

- **Task 4:** 4 rows

## 8. Output:

**Task 1**

```
****
****
****
****
```

**Task 2**

```
*
**
***
****
```

**Task 3**

```
1
1 2
1 2 3
1 2 3 4
```

**Task 4**

```
****
***
**
*
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is a nested loop?
Q2. In a pattern program, which loop usually controls the rows and which controls the columns?
Q3. What is the difference between print() and println() in pattern printing?
Q4. How will you change the right triangle program to print an upside-down triangle?
Q5. How many times does the inner statement run for a 4 x 4 square?

# EXPERIMENT NO. 4

Date of Performance:- Course: BCS351

## 1. Experiment Title:

- Writing Functions

## 2. Aim:

- To write functions (methods) with parameters and return values, and call them from main().

## 3. Software / Tools Required:

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

## 4. Theory:

- A function (method) is a named block of code that performs one task. It is written once and can be called many times, which avoids repeating code.
- Parameters are the values given to a function, and the return statement sends a result back to the caller. A static method can be called directly from main() without creating an object.
- Combinations: nCr = n! / (r! x (n-r)!). Permutations: nPr = n! / (n-r)!. Both can be calculated by calling a factorial function.

## 5. Algorithm / Procedure:

**Task 1: Write fact(n) and nCr(n, r) and print 5C2.**

1. Write fact(n): multiply 1 x 2 x ... x n in a loop and return it.
2. Write nCr(n, r): return fact(n) / (fact(r) * fact(n - r)).
3. In main() call nCr(5, 2) and print it.
4. Stop.

**Task 2: Write fact(n) and nPr(n, r) and print 5P2.**

1. Write fact(n) using a loop.
2. Write nPr(n, r): return fact(n) / fact(n - r).
3. In main() call nPr(5, 2) and print it.
4. Stop.

**Task 3: Write maxOf(a, b) that returns the larger of two numbers.**

1. Write maxOf(a, b): if a > b return a, else return b.
2. In main() call maxOf(14, 9) and print the result.
3. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) {
            f = f * i;
        }
        return f;
    }

    static int nCr(int n, int r) {
        return fact(n) / (fact(r) * fact(n - r));
    }

    public static void main(String[] args) {
        System.out.println("5C2 = " + nCr(5, 2));
    }
}
```

**Task 2**

```
public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) {
            f = f * i;
        }
        return f;
    }

    static int nPr(int n, int r) {
        return fact(n) / fact(n - r);
    }

    public static void main(String[] args) {
        System.out.println("5P2 = " + nPr(5, 2));
    }
}
```

**Task 3**

```
public class Main {
    static int maxOf(int a, int b) {
        if (a > b) {
            return a;
        } else {
            return b;
        }
    }

    public static void main(String[] args) {
        System.out.println("Larger: " + maxOf(14, 9));
    }
}
```

## 7. Input:

- **Task 1:** n = 5, r = 2
- **Task 2:** n = 5, r = 2

- **Task 3:** a = 14, b = 9

## 8. Output:

**Task 1**

```
5C2 = 10
```

**Task 2**

```
5P2 = 20
```

**Task 3**

```
Larger: 14
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is a method and why do we use it?
Q2. What is the difference between a parameter and a return value?
Q3. Why is the keyword static used before the methods in this program?
Q4. Write the formulas for nCr and nPr.
Q5. Can a method call another method? Give an example from this experiment.

# EXPERIMENT NO. 5

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Arrays: Traversal and Counting

**2. Aim:**

- To create an array, visit every element with a loop, and add up or count the values.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- An array stores a fixed number of values of the same type in consecutive memory locations. Elements are accessed with an index that starts at 0, and arr.length gives the number of elements.
- Traversal means visiting every element once, normally with a for loop from index 0 to arr.length - 1.
- While traversing, we can keep a running total (sum), keep the largest value seen so far, or count elements that satisfy a condition.

**5. Algorithm / Procedure:**

**Task 1: Print every element of arr = {3, 4, 5} with its index.**

1. Create int[] arr = {3, 4, 5}.
2. Loop i from 0 to arr.length - 1.
3. Print "Index i: arr[i]".
4. Stop.

**Task 2: Find the sum and the largest value of {12, 45, 7, 33, 21}.**

1. Create the array. Set sum = 0 and largest = arr[0].
2. Loop through the array: add each element to sum; if it is greater than largest, update largest.
3. Print sum and largest.
4. Stop.

**Task 3: Count the values greater than 10 in {5, 12, 2, 14, 7}.**

1. Create the array. Set count = 0.
2. Loop through the array: if arr[i] > 10, increase count by 1.
3. Print count.
4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {3, 4, 5};
        for (int i = 0; i < arr.length; i++) {
            System.out.println("Index " + i + ": " + arr[i]);
        }
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {12, 45, 7, 33, 21};
        int sum = 0;
        int largest = arr[0];
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
            if (arr[i] > largest) {
                largest = arr[i];
            }
        }
        System.out.println("Sum: " + sum);
        System.out.println("Largest: " + largest);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 12, 2, 14, 7};
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > 10) {
                count++;
            }
        }
        System.out.println("Count greater than 10: " + count);
    }
}
```

## 7. Input:

- **Task 1:** arr = {3, 4, 5}
- **Task 2:** arr = {12, 45, 7, 33, 21}
- **Task 3:** arr = {5, 12, 2, 14, 7}

## 8. Output:

**Task 1**

```
Index 0: 3
Index 1: 4
Index 2: 5
```

**Task 2**

```
Sum: 118
Largest: 45
```

**Task 3**

```
Count greater than 10: 2
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is an array? What is the index of its first element?
Q2. What does arr.length return?
Q3. What happens if we access arr[5] in an array of size 3?
Q4. Why do we start the largest variable with arr[0]?
Q5. What is traversal of an array?

# EXPERIMENT NO. 6

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Arrays: Reverse and Duplicates

**2. Aim:**

- To print an array in reverse order, find the smallest value, and check an array for repeated values.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- To print an array in reverse order, a loop starts from the last index (length - 1) and moves down to index 0.
- To find the smallest value, assume the first element is the smallest, then compare it with every other element and update it (and its index) whenever a smaller value is found.
- To find a repeated value, nested loops compare each element with every element after it. If two elements are equal, a duplicate exists.

**5. Algorithm / Procedure:**

**Task 1: Print {10, 20, 30, 40, 50} in reverse order.**

1. Create the array.
2. Loop i from arr.length - 1 down to 0.
3. Print arr[i] on each line.
4. Stop.

**Task 2: Find the smallest value and its index in {2, 5, 1, 8, 3, 4}.**

1. Create the array. Set smallest = arr[0] and index = 0.
2. Loop from i = 1: if arr[i] < smallest, set smallest = arr[i] and index = i.
3. Print smallest and index.
4. Stop.

**Task 3: Check {4, 9, 2, 9, 7} for a repeated value using nested loops.**

1. Create the array. Set found = false.
2. Outer loop i from 0; inner loop j from i + 1.
3. If arr[i] == arr[j], print it as the repeated value, set found = true and stop searching.
4. If nothing was found, print "No repeated value".
5. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        for (int i = arr.length - 1; i >= 0; i--) {
            System.out.println(arr[i]);
        }
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 5, 1, 8, 3, 4};
        int smallest = arr[0];
        int index = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < smallest) {
                smallest = arr[i];
                index = i;
            }
        }
        System.out.println("Smallest: " + smallest + " at index " + index);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 9, 2, 9, 7};
        boolean found = false;
        for (int i = 0; i < arr.length && !found; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println("Repeated value: " + arr[i]);
                    found = true;
                    break;
                }
            }
        }
        if (!found) {
            System.out.println("No repeated value");
        }
    }
}
```

## 7. Input:

- **Task 1:** arr = {10, 20, 30, 40, 50}
- **Task 2:** arr = {2, 5, 1, 8, 3, 4}
- **Task 3:** arr = {4, 9, 2, 9, 7}

## 8. Output:

**Task 1**

```
50
40
30
20
10
```

**Task 2**

```
Smallest: 1 at index 2
```

**Task 3**

```
Repeated value: 9
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. How do you print an array in reverse order?
Q2. Why do we start the inner loop from i + 1 while searching for duplicates?
Q3. What is the purpose of the boolean variable found?
Q4. What is the time complexity of the duplicate check using nested loops?
Q5. How can the smallest value be found without sorting the array?

# EXPERIMENT NO. 7

Date of Performance:- Course: BCS351

## 1. Experiment Title:

- Counting Steps and Complexity

## 2. Aim:

- To count the steps a program takes, and to write its time complexity as a comment.

## 3. Software / Tools Required:

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

## 4. Theory:

- Time complexity describes how the number of steps of a program grows as the input size n grows. It is written in Big-O notation.
- A single loop that runs n times takes O(n) steps. Two nested loops that both run n times take n x n steps, so $O(n^2)$.
- A loop that halves its value in every step, such as n = n / 2, needs about log2(n) steps, so its complexity is O(log n). For n = 1024 this is 10 steps.

## 5. Algorithm / Procedure:

**Task 1: Count the steps of a single loop from 1 to 10.**

1. Set steps = 0.
2. Loop i from 1 to 10 and increase steps by 1 each time.
3. Print steps. Complexity is O(n).
4. Stop.

**Task 2: Count the steps of two nested loops that both run from 1 to 10.**

1. Set steps = 0.
2. Outer loop i from 1 to 10; inner loop j from 1 to 10.
3. Increase steps by 1 in the inner loop body.
4. Print steps. Complexity is $O(n^2)$.
5. Stop.

**Task 3: Count how many times n = 1024 can be halved until it becomes 1.**

1. Store 1024 in n. Set steps = 0.
2. While n > 1: set n = n / 2 and increase steps by 1.
3. Print steps. Complexity is O(log n).
4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n) - the loop runs n times (n = 10)
        for (int i = 1; i <= 10; i++) {
            steps++;
        }
        System.out.println("Steps: " + steps);
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n^2) - the inner body runs n * n times (n = 10)
        for (int i = 1; i <= 10; i++) {
            for (int j = 1; j <= 10; j++) {
                steps++;
            }
        }
        System.out.println("Steps: " + steps);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int n = 1024;
        int steps = 0;
        // Time complexity: O(log n) - n is halved in every step
        while (n > 1) {
            n = n / 2;
            steps++;
        }
        System.out.println("Halving steps: " + steps);
    }
}
```

## 7. Input:

- **Task 1:** Loop from 1 to 10 (n = 10)
- **Task 2:** Both loops from 1 to 10 (n = 10)
- **Task 3:** n = 1024

## 8. Output:

**Task 1**

```
Steps: 10
```

**Task 2**

```
Steps: 100
```

**Task 3**

```
Halving steps: 10
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is time complexity?
Q2. What is the time complexity of a single loop and of two nested loops?
Q3. Why is the halving loop O(log n)?
Q4. What does Big-O notation represent?
Q5. Arrange O(n), O(log n) and $O(n^2)$ from fastest to slowest.

# EXPERIMENT NO. 8

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Matrices: Transpose and Row Sums

**2. Aim:**

- To create two-dimensional arrays, visit them row by row, and build a new matrix from one.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- A two-dimensional array (matrix) stores values in rows and columns. m[i][j] is the element in row i and column j. m.length gives the number of rows and m[0].length gives the number of columns.
- Nested loops are used to visit all elements: the outer loop moves over rows and the inner loop over columns.
- The transpose of a matrix is obtained by swapping its rows and columns, so t[j][i] = m[i][j]. The row sum is the total of all elements of a row.

**5. Algorithm / Procedure:**

**Task 1: Fill a 3 x 4 matrix with the values 1 to 12 and print it.**

1. Create int[][] m = new int[3][4] and set value = 1.
2. Nested loops: store value in m[i][j] and increase value.
3. Nested loops again: print m[i][j] followed by a space; new line after each row.
4. Stop.

**Task 2: Find and print the transpose of a 3 x 3 matrix.**

1. Create the matrix {{1,2,3},{4,5,6},{7,8,9}} and an empty 3 x 3 matrix t.
2. Nested loops: t[j][i] = m[i][j].
3. Print t row by row.
4. Stop.

**Task 3: Print the sum of each row of a 3 x 3 matrix.**

1. Create the matrix.
2. For each row i: set sum = 0 and add m[i][j] for all columns j.
3. Print "Row (i+1) sum: sum".
4. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        int[][] m = new int[3][4];
        int value = 1;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                m[i][j] = value;
                value++;
            }
        }
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                System.out.print(m[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

### Task 2

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int[][] t = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                t[j][i] = m[i][j];
            }
        }
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(t[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

### Task 3

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        for (int i = 0; i < 3; i++) {
            int sum = 0;
            for (int j = 0; j < 3; j++) {
                sum = sum + m[i][j];
            }
            System.out.println("Row " + (i + 1) + " sum: " + sum);
        }
    }
}
```

## 7. Input:

- **Task 1:** Matrix of 3 rows and 4 columns filled with 1 to 12
- **Task 2:** m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
- **Task 3:** m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}

## 8. Output:

**Task 1**

```
1 2 3 4
5 6 7 8
9 10 11 12
```

**Task 2**

```
1 4 7
2 5 8
3 6 9
```

**Task 3**

```
Row 1 sum: 6
Row 2 sum: 15
Row 3 sum: 24
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What is a two-dimensional array? How is it declared?
Q2. How do you find the number of rows and columns of a matrix?
Q3. What is the transpose of a matrix?
Q4. Why are two nested loops needed to print a matrix?
Q5. What is the time complexity of printing an n x n matrix?

# EXPERIMENT NO. 9

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Matrices: Rows and Columns

**2. Aim:**

- To visit the rows and columns of a matrix with nested loops to print, add and count its values.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- In a matrix, the first row is m[0] and the last row is m[m.length - 1]. A single loop over the column index prints one full row.
- To add the values of a column, the column index is fixed in the outer loop and the row index changes in the inner loop.
- A counter variable can be increased inside nested loops whenever an element satisfies a condition, for example m[i][j] > 5.

**5. Algorithm / Procedure:**

**Task 1: Print the first row and the last row of a 3 x 3 matrix.**

1. Create the matrix.
2. Print "First row:" and loop over columns j printing m[0][j].
3. Print "Last row:" and loop over columns j printing m[2][j].
4. Stop.

**Task 2: Print the sum of each column of a 3 x 3 matrix.**

1. Create the matrix.
2. Outer loop over columns j: set sum = 0.
3. Inner loop over rows i: add m[i][j] to sum.
4. Print "Column (j+1) sum: sum".
5. Stop.

**Task 3: Count the values greater than 5 in a 3 x 3 matrix.**

1. Create the matrix. Set count = 0.
2. Nested loops: if m[i][j] > 5, increase count.
3. Print count.
4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

        System.out.print("First row:");
        for (int j = 0; j < 3; j++) {
            System.out.print(" " + m[0][j]);
        }
        System.out.println();

        System.out.print("Last row:");
        for (int j = 0; j < 3; j++) {
            System.out.print(" " + m[2][j]);
        }
        System.out.println();
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        for (int j = 0; j < 3; j++) {
            int sum = 0;
            for (int i = 0; i < 3; i++) {
                sum = sum + m[i][j];
            }
            System.out.println("Column " + (j + 1) + " sum: " + sum);
        }
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int count = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (m[i][j] > 5) {
                    count++;
                }
            }
        }
        System.out.println("Values greater than 5: " + count);
    }
}
```

## 7. Input:

- **Task 1:** m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
- **Task 2:** m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
- **Task 3:** m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}

## 8. Output:

### Task 1

```
First row: 1 2 3
Last row: 7 8 9
```

### Task 2

```
Column 1 sum: 12
Column 2 sum: 15
Column 3 sum: 18
```

### Task 3

```
Values greater than 5: 4
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. How do you access the last row of a matrix?
Q2. How does the loop order change when finding column sums instead of row sums?
Q3. What is the index of the element 6 in the matrix used here?
Q4. How many elements are checked in a 3 x 3 matrix when counting values?
Q5. Can the rows of a Java 2D array have different lengths?

# EXPERIMENT NO. 10

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Switch and ArrayList

**2. Aim:**

- To choose a branch with a switch statement, and to store and change a list of numbers in an ArrayList.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- The switch statement selects one block of code from many choices by comparing a value with case labels. break ends a case, and default runs when no case matches.
- ArrayList is a resizable list from java.util. Unlike an array, its size grows and shrinks automatically. It stores objects, so int values are stored as Integer.
- Important methods: add(value), get(index), size(), set(index, value) and remove(index). Printing an ArrayList shows its elements inside square brackets.

**5. Algorithm / Procedure:**

**Task 1: Print the name of the day for day = 3 using a switch statement.**

1. Store 3 in day.
2. switch(day): case 1 prints Monday, case 2 prints Tuesday, case 3 prints Wednesday.
3. default prints "Invalid day".
4. Stop.

**Task 2: Create an ArrayList, add 10, 20, 30, 40 and print the list, size and item at index 2.**

1. Import java.util.ArrayList and create ArrayList<Integer> list.
2. Add 10, 20, 30 and 40 using add().
3. Print the list, list.size() and list.get(2).
4. Stop.

**Task 3: Change index 1 to 25 using set() and delete index 3 using remove().**

1. Create the list with 10, 20, 30, 40.
2. list.set(1, 25) changes the second element.
3. list.remove(3) deletes the fourth element.
4. Print the list and its size.
5. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        int day = 3;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

### Task 2

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);

        System.out.println(list);
        System.out.println("Size: " + list.size());
        System.out.println("Item at index 2: " + list.get(2));
    }
}
```

### Task 3

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);

        list.set(1, 25);
        list.remove(3);

        System.out.println(list);
        System.out.println("Size: " + list.size());
    }
}
```

### 7. Input:

- **Task 1:** day = 3
- **Task 2:** ArrayList values 10, 20, 30, 40
- **Task 3:** ArrayList values 10, 20, 30, 40; set(1, 25) and remove(3)

### 8. Output:

**Task 1**

```
Wednesday
```

**Task 2**

```
[10, 20, 30, 40]
Size: 4
Item at index 2: 30
```

**Task 3**

```
[10, 25, 30]
Size: 3
```

### 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

### 10. Viva-Voce Questions:

Q1. Why is break used in a switch statement?
Q2. What is the use of the default case?
Q3. What is the difference between an array and an ArrayList?
Q4. Which package contains ArrayList?
Q5. What is the difference between set() and add() in an ArrayList?

# EXPERIMENT NO. 11

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Bubble Sort and Selection Sort

**2. Aim:**

- To sort small arrays with bubble sort and selection sort, and to count the swaps.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- Bubble sort repeatedly compares neighbouring elements and swaps them if they are in the wrong order. After each pass the largest remaining value moves to its correct place at the end.
- Selection sort finds the smallest value in the unsorted part of the array and swaps it with the first element of that part. It repeats this for every position.
- Both algorithms use two nested loops and take $O(n^2)$ time in the worst case. Selection sort makes at most n - 1 swaps.

**5. Algorithm / Procedure:**

**Task 1: Sort {5, 1, 4, 2, 8} in increasing order using bubble sort.**

1. Loop i from 0 to n - 2 (passes).
2. Inner loop j from 0 to n - i - 2: if arr[j] > arr[j + 1], swap them.
3. After all passes the array is sorted; print it as [a, b, c, ...].
4. Stop.

**Task 2: Sort {64, 25, 12, 22, 11} in increasing order using selection sort.**

1. Loop i from 0 to n - 2; set min = i.
2. Inner loop j from i + 1 to n - 1: if arr[j] < arr[min], set min = j.
3. Swap arr[i] with arr[min].
4. Print the sorted array.
5. Stop.

**Task 3: Count the swaps made by bubble sort on {5, 1, 4, 2, 8}.**

1. Set swaps = 0.
2. Run bubble sort; each time two values are swapped, increase swaps by 1.
3. Print swaps.
4. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        System.out.print("Sorted: [");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i]);
            if (i < n - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}
```

### Task 2

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int min = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[min]) {
                    min = j;
                }
            }
            int temp = arr[min];
            arr[min] = arr[i];
            arr[i] = temp;
        }

        System.out.print("Sorted: [");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i]);
            if (i < n - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        int n = arr.length;
        int swaps = 0;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swaps++;
                }
            }
        }

        System.out.println("Swaps: " + swaps);
    }
}
```

## 7. Input:

- **Task 1:** arr = {5, 1, 4, 2, 8}
- **Task 2:** arr = {64, 25, 12, 22, 11}
- **Task 3:** arr = {5, 1, 4, 2, 8}

## 8. Output:

**Task 1**

```
Sorted: [1, 2, 4, 5, 8]
```

**Task 2**

```
Sorted: [11, 12, 22, 25, 64]
```

**Task 3**

```
Swaps: 4
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. How does bubble sort work?
Q2. How does selection sort differ from bubble sort?
Q3. What is the worst-case time complexity of both algorithms?
Q4. Why do we need a temporary variable while swapping two values?
Q5. How many swaps does selection sort make at most for n elements?

# EXPERIMENT NO. 12

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Insertion Sort

**2. Aim:**

- To sort small arrays with insertion sort and to count how many values are shifted.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- Insertion sort builds the sorted array one element at a time, like arranging playing cards in the hand. Each new element (key) is inserted into its correct place among the already sorted elements on its left.
- Elements larger than the key are shifted one place to the right to make room. The key is then placed in the gap.
- Insertion sort takes $O(n^2)$ time in the worst case (reverse order) and O(n) in the best case (already sorted). For {6, 5, 4, 3, 2, 1} the number of shifts is 5 + 4 + 3 + 2 + 1 = 15.

**5. Algorithm / Procedure:**

**Task 1: Sort {7, 3, 5, 1} using insertion sort.**

1. For i = 1 to n - 1: key = arr[i], j = i - 1.
2. While j >= 0 and arr[j] > key: arr[j + 1] = arr[j] and j--.
3. Place key at arr[j + 1].
4. Print the sorted array.
5. Stop.

**Task 2: Sort {14, 3, 9, 21, 6} using insertion sort.**

1. Apply the same insertion sort steps on the array {14, 3, 9, 21, 6}.
2. Print the sorted array.
3. Stop.

**Task 3: Count the shifts made while sorting {6, 5, 4, 3, 2, 1}.**

1. Set shifts = 0.
2. Run insertion sort; each time arr[j] is moved to arr[j + 1], increase shifts by 1.
3. Print shifts.
4. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {7, 3, 5, 1};
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }

        System.out.print("Sorted: [");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i]);
            if (i < n - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}
```

### Task 2

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {14, 3, 9, 21, 6};
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }

        System.out.print("Sorted: [");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i]);
            if (i < n - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 5, 4, 3, 2, 1};
        int n = arr.length;
        int shifts = 0;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                shifts++;
                j--;
            }
            arr[j + 1] = key;
        }

        System.out.println("Shifts: " + shifts);
    }
}
```

### 7. Input:

- **Task 1:** arr = {7, 3, 5, 1}
- **Task 2:** arr = {14, 3, 9, 21, 6}
- **Task 3:** arr = {6, 5, 4, 3, 2, 1}

### 8. Output:

**Task 1**

```
Sorted: [1, 3, 5, 7]
```

**Task 2**

```
Sorted: [3, 6, 9, 14, 21]
```

**Task 3**

```
Shifts: 15
```

### 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

### 10. Viva-Voce Questions:

Q1. Explain the working of insertion sort with an example.
Q2. What is the key in insertion sort?
Q3. What is the best-case and worst-case time complexity of insertion sort?
Q4. Why does a reverse-sorted array need the maximum number of shifts?
Q5. Is insertion sort suitable for small arrays? Why?

# EXPERIMENT NO. 13

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Strings: Methods and Palindromes

**2. Aim:**

- To use common String methods, reverse a word, check a palindrome and print substrings.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- A String is an object that stores a sequence of characters. Strings are immutable, which means their value cannot be changed after creation.
- Common methods: length() gives the number of characters, indexOf(ch) gives the first position of a character (counting from 0), toUpperCase() converts to capital letters, charAt(i) gives the character at index i and substring(start, end) gives a part of the string.
- A palindrome reads the same from both ends (for example racecar). A string can be reversed by adding its characters one by one from the last index to the first.

**5. Algorithm / Procedure:**

**Task 1: Print the length, the position of g and the upper-case form of "Programming".**

1. Store "Programming" in word.
2. Print word.length(), word.indexOf('g') and word.toUpperCase().
3. Stop.

**Task 2: Reverse the word "Java" using charAt and + in a loop.**

1. Store "Java" in word and set reversed = "".
2. Loop i from word.length() - 1 down to 0: reversed = reversed + word.charAt(i).
3. Print reversed.
4. Stop.

**Task 3: Check whether "racecar" is a palindrome.**

1. Store "racecar" in word and build its reverse in rev.
2. If word equals rev, print "is a palindrome"; otherwise print "is not a palindrome".
3. Stop.

**Task 4: Print every substring of "abc".**

1. Store "abc" in word.
2. Outer loop i from 0 to length - 1 (start of substring).
3. Inner loop j from i + 1 to length: print word.substring(i, j).
4. Stop.

## 6. Program / Queries / Commands:

### Task 1

```
public class Main {
    public static void main(String[] args) {
        String word = "Programming";
        System.out.println("Length: " + word.length());
        System.out.println("Position of g: " + word.indexOf('g'));
        System.out.println("Upper case: " + word.toUpperCase());
    }
}
```

### Task 2

```
public class Main {
    public static void main(String[] args) {
        String word = "Java";
        String reversed = "";
        for (int i = word.length() - 1; i >= 0; i--) {
            reversed = reversed + word.charAt(i);
        }
        System.out.println("Reversed: " + reversed);
    }
}
```

### Task 3

```
public class Main {
    public static void main(String[] args) {
        String word = "racecar";
        String rev = "";
        for (int i = word.length() - 1; i >= 0; i--) {
            rev = rev + word.charAt(i);
        }
        if (word.equals(rev)) {
            System.out.println(word + " is a palindrome");
        } else {
            System.out.println(word + " is not a palindrome");
        }
    }
}
```

### Task 4

```
public class Main {
    public static void main(String[] args) {
        String word = "abc";
        for (int i = 0; i < word.length(); i++) {
            for (int j = i + 1; j <= word.length(); j++) {
                System.out.println(word.substring(i, j));
            }
        }
    }
}
```

## 7. Input:

- **Task 1:** word = "Programming"
- **Task 2:** word = "Java"
- **Task 3:** word = "racecar"

- **Task 4:** word = "abc"

## 8. Output:

**Task 1**

```
Length: 11
Position of g: 3
Upper case: PROGRAMMING
```

**Task 2**

```
Reversed: avaJ
```

**Task 3**

```
racecar is a palindrome
```

**Task 4**

```
a
ab
abc
b
bc
c
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What does it mean that a String is immutable?
Q2. What is the difference between length() of a String and length of an array?
Q3. Why should equals() be used instead of == to compare strings?
Q4. What is a palindrome? Give two examples.
Q5. What does substring(1, 3) return for the string "abcd"?

# EXPERIMENT NO. 14

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- Characters: Counting and Positions

**2. Aim:**

- To use charAt to count vowels and a letter, and to print each letter with its position.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- charAt(i) returns the character at position i of a String, where positions start at 0 and end at length() - 1. A char is written inside single quotes, for example 'g'.
- To count characters, a loop visits every position and a counter is increased whenever the character satisfies a condition, such as being a vowel (a, e, i, o, u) or being equal to a given letter.
- Characters can be compared with == and combined conditions can be written with the || (OR) operator.

**5. Algorithm / Procedure:**

**Task 1: Count the vowels in "programming".**

1. Store "programming" in s and set count = 0.
2. Loop i over the string: ch = s.charAt(i).
3. If ch is a, e, i, o or u, increase count.
4. Print count.
5. Stop.

**Task 2: Count how many times the letter g appears in "programming".**

1. Store "programming" in s and set count = 0.
2. Loop over the string: if s.charAt(i) == 'g', increase count.
3. Print count.
4. Stop.

**Task 3: Print every letter of "hello" with its position.**

1. Store "hello" in s.
2. Loop i from 0 to s.length() - 1.
3. Print i, a colon, a space and s.charAt(i).
4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                count++;
            }
        }
        System.out.println("Vowels: " + count);
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == 'g') {
                count++;
            }
        }
        System.out.println("Count of g: " + count);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        String s = "hello";
        for (int i = 0; i < s.length(); i++) {
            System.out.println(i + ": " + s.charAt(i));
        }
    }
}
```

## 7. Input:

- **Task 1:** s = "programming"
- **Task 2:** s = "programming"
- **Task 3:** s = "hello"

## 8. Output:

**Task 1**

```
Vowels: 3
```

**Task 2**

```
Count of g: 2
```

**Task 3**

```
0: h
1: e
2: l
3: l
4: o
```

## 9. Result:

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

## 10. Viva-Voce Questions:

Q1. What does charAt() return and what is the index of the first character?
Q2. What is the difference between 'a' and "a" in Java?
Q3. Which operator is used to combine several conditions with OR?
Q4. How would you count uppercase vowels as well?
Q5. What happens if charAt() is given an index equal to the string length?

# EXPERIMENT NO. 15

Date of Performance:- Course: BCS351

**1. Experiment Title:**

- StringBuilder Methods

**2. Aim:**

- To build and change text with StringBuilder, using append, insert and deleteCharAt.

**3. Software / Tools Required:**

- Programming language: Java. Software: Java Development Kit (JDK 21) and an IDE such as VS Code, IntelliJ IDEA or Eclipse (or Command Prompt / Terminal). Operating system: Windows / Linux.

**4. Theory:**

- String objects cannot be changed. Every change creates a new String. StringBuilder is a mutable class: its content can be changed in the same object, which is faster when text is modified many times.
- append(x) adds text or a number at the end, insert(index, text) adds text at the given position, deleteCharAt(index) removes the character at the given position, length() gives the number of characters and toString() converts it to a String.
- System.out.println() can print a StringBuilder directly because it displays its text content.

**5. Algorithm / Procedure:**

**Task 1: Start with "Hello", append " World" and print the text and its length.**

1. Create StringBuilder sb = new StringBuilder("Hello").
2. sb.append(" World").
3. Print sb and sb.length().
4. Stop.

**Task 2: Append the numbers 1 to 5 with commas and remove the last comma.**

1. Create an empty StringBuilder sb.
2. Loop i from 1 to 5: append i and then ",".
3. Remove the last comma using sb.deleteCharAt(sb.length() - 1).
4. Print sb.
5. Stop.

**Task 3: Start with "Java" and insert "Hello " at index 0.**

1. Create StringBuilder sb = new StringBuilder("Java").
2. sb.insert(0, "Hello ").
3. Print sb.
4. Stop.

## 6. Program / Queries / Commands:

**Task 1**

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        System.out.println(sb);
        System.out.println("Length: " + sb.length());
    }
}
```

**Task 2**

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 5; i++) {
            sb.append(i);
            sb.append(",");
        }
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb);
    }
}
```

**Task 3**

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.insert(0, "Hello ");
        System.out.println(sb);
    }
}
```

## 7. Input:

- **Task 1:** Text "Hello", appended text " World"
- **Task 2:** Numbers 1 to 5
- **Task 3:** Text "Java", inserted text "Hello " at index 0

## 8. Output:

**Task 1**

```
Hello World
Length: 11
```

**Task 2**

```
1,2,3,4,5
```

**Task 3**

```
Hello Java
```

**9. Result:**

- The programs of this experiment were executed successfully and the output was verified. Thus the aim of the experiment was achieved.

**10. Viva-Voce Questions:**

Q1. What is the difference between String and StringBuilder?
Q2. Why is StringBuilder preferred when text is changed many times?
Q3. What does insert(0, "Hello ") do?
Q4. How do you remove the last character of a StringBuilder?
Q5. How can a StringBuilder be converted into a String?